

Employee Payroll System

Python Programming Project

Student: GIL Antony

Course: Python Programming

Professor: Marcin Pawlik

Academic Year: 2025/2026

University: Politechnika Krakowska im. Tadeusza Kościuszki



Politechnika Krakowska
im. Tadeusza Kościuszki

Table des matières

I.	PROJECT OVERVIEW	3
A.	Objectives	3
B.	Main Features	3
II.	PROJECT STRUCTURE	4
III.	UML CLASS DIAGRAM	5
IV.	IMPLEMENTATION DETAILS	6
A.	Object-Oriented Programming Concepts	6
B.	Technical Implementation	7
V.	Program Demonstration	8
A.	Employee Types Examples	8
B.	Interactive Menu	8
C.	Sample Output	9
VI.	SOURCE CODE REPOSITORY	10
VII.	CONCLUSION	11

I. PROJECT OVERVIEW

A. Objectives

This project implements an employee payroll management system in Python that demonstrates object-oriented programming principles. The system manages different types of employees, calculates their monthly salaries based on their employment type, and provides a flexible interface for company operations.

The main goal is to create a functional application that showcases:

- Proper class design and encapsulation
- Inheritance and method overriding
- Polymorphism in action
- List management and iteration
- Input validation and error handling
- Interactive user interface

B. Main Features

Employee Management System

- Three employee types: Full-time, Part-time, and Apprentice
- Unique identification for each employee
- Multiple methods for adding employees to the company

Salary Calculation

- Type-specific salary computation:
 - Full-time: Fixed monthly salary
 - Part-time: Hours worked $\times$ hourly rate
 - Apprentice: 70% of minimum wage (1380 €)

Interactive Interface

- Menu-driven user interaction
- Input validation with error handling
- Real-time operation feedback

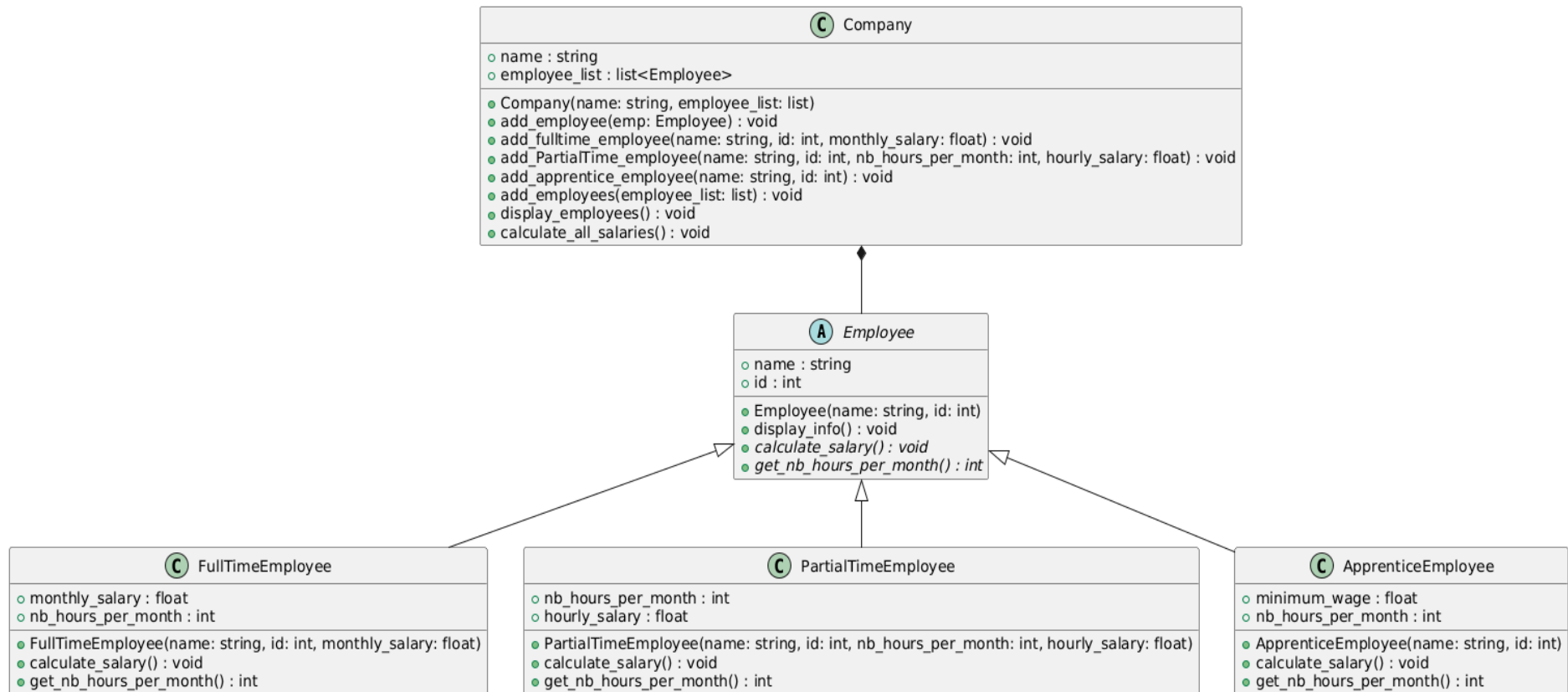
II. PROJECT STRUCTURE

The project is organized into four modular Python files:

File	Description
employee.py	Contains the Employee base class and three concrete employee types (FullTimeEmployee, PartialTimeEmployee, ApprenticeEmployee)
company.py	Manages the Company class with employee lists and salary calculation operations
tools.py	Provides utility functions for input validation and interactive menu management
Project.py	Main entry point that creates demo data and launches the interactive system

This modular organization separates concerns and improves maintainability.

III. UML CLASS DIAGRAM



Relationships:

- Employee serves as an abstract base class with methods that must be implemented by child classes
- The three employee types inherit from Employee and provide specific implementations
- Company maintains a composition relationship with Employee (has-a relationship)

IV. IMPLEMENTATION DETAILS

A. Object-Oriented Programming Concepts

Inheritance

The three employee classes inherit from the Employee base class:

```
class FullTimeEmployee(Employee):  
    # Full-time employee class: works 156 hours per month
```

```
class PartialTimeEmployee(Employee):  
    # Part-time employee class: variable hours and hourly pay
```

```
class ApprenticeEmployee(Employee):  
    # Apprentice class: earns 70% of minimum wage
```

The `super()` function calls the parent class constructor to initialize common attributes.

```
super().__init__(name, id)
```

Polymorphism

Each employee type implements `calculate_salary()` differently:

FullTimeEmployee

```
def calculate_salary(self):  
    # Calculate salary for full-time employee  
    self.salary = self.monthly_salary  
    print(f"The salary of {self.name} (FullTimeEmployee) is {self.salary:.2f} €")
```

PartialTimeEmployee

```
def calculate_salary(self):  
    # Calculate salary = hours * hourly rate  
    self.salary = self.nb_hours_per_month * self.hourly_salary  
    print(f"The salary of {self.name} (PartialTimeEmployee) is {self.salary:.2f} €")
```

ApprenticeEmployee

```
def calculate_salary(self):  
    # Calculate salary = minimum_wage * 0.7  
    self.salary = self.minimum_wage * 0.7  
    print(f"The salary of {self.name} (ApprenticeEmployee) is {self.salary:.2f} €")
```

When iterating through employees, the correct calculation method is called automatically:

```
for e in self.employee_list:  
    e.calculate_salary()
```

Calls the appropriate method for each type

B. Technical Implementation

Input Validation

The tools module provides robust input validation:

```
def input_int(prompt: str):
```

```
def input_int(prompt: str):  
    # Get integer input from user  
    while True:  
        try:  
            return int(input(prompt))  
        except ValueError:  
            print("Please enter a valid integer.")
```

This prevents invalid data from entering the system and improves user experience.

List Management

The Company class uses Python lists to store employees:

```
class Company:  
    # Company class to manage employees  
  
    def __init__(self, name: str, employee_list=None):  
        # Initialize company with name and optional list of employees  
        self.name = name  
        self.employee_list = employee_list if employee_list is not None else []
```

Type Hinting

Type hints improve code readability:

```
def add_employee(self, emp: Employee):  
    # Add an employee object to the company  
    self.employee_list.append(emp)
```

V. Program Demonstration

A. Employee Types Examples

Full-Time Employee:

```
emp = FullTimeEmployee("Cocotte", 1402, 3000)
```

```
The salary of Coco (PartialTimeEmployee) is 2340.00 €  
...for 130 hours worked
```

Part-Time Employee:

```
emp = PartialTimeEmployee("Jérôme", 72, 120, 20)
```

```
The salary of Jérôme (PartialTimeEmployee) is 2400.00 €  
...for 120 hours worked
```

Apprentice Employee:

```
emp = ApprenticeEmployee("Tata", 18)
```

```
The salary of Tata (ApprenticeEmployee) is 966.00 €  
...for 151 hours worked
```

B. Interactive Menu

The program provides a menu-driven interface:

```
----- Menu -----  
1. Add a full-time employee  
2. Add a part-time employee  
3. Add an apprentice  
4. Display employee list  
5. Calculate all salaries  
6. Quit  
Choice: █
```

Example interaction:

```
Choice: 1  
Name: Alice  
ID (integer): 100  
Monthly salary: 2500  
Full-time employee added.
```

C. Sample Output

Display Employees:

```
***** List of employees at Gil and Co.: *****
I am Jérôme and I have id 72
I am Toto and I have id 142
I am Cocotte and I have id 1402
I am Coco and I have id 1408
I am Titi and I have id 15
I am Tata and I have id 18
*****
```

Calculate Salaries:

```
***** Calculating employee salaries: *****
The salary of Jérôme (PartialTimeEmployee) is 2400.00 €
...for 120 hours worked
The salary of Toto (FullTimeEmployee) is 1500.00 €
...for 156 hours worked
The salary of Cocotte (FullTimeEmployee) is 3000.00 €
...for 156 hours worked
The salary of Coco (PartialTimeEmployee) is 2340.00 €
...for 130 hours worked
The salary of Titi (FullTimeEmployee) is 2000.00 €
...for 156 hours worked
The salary of Tata (ApprenticeEmployee) is 966.00 €
...for 151 hours worked
*****
```

VI. SOURCE CODE REPOSITORY

The complete source code is available on GitHub at
<https://github.com/AntonyGil1999/Employee-Payroll-System>

Installation of the Source Code from GitHub:

1. Create an empty folder on your computer.
2. Open Git Bash (or terminal) and navigate to this folder.
3. Run the following command to clone the repository:

git clone <https://github.com/AntonyGil1999/Employee-Payroll-System.git>

The repository includes:

- company.py
- employee.py
- tools.py
- Project.py
- README.md
- UML Projet Python.png
- Poster project.jpeg
- System_Anatomy_A_Payroll_Case_Study.pdf

Execution: python Project.py

Requirements:

- Python 3.6 or higher

No external libraries required (uses only standard Python)

VII. CONCLUSION

This project explores the core principles of object-oriented programming in Python through a practical payroll management system.

Key Concepts Practiced

The implementation allowed me to practice:

- Designing class hierarchies with inheritance
- Implementing polymorphic behavior through method overriding
- Managing collections with lists and iteration
- Validating user input with exception handling
- Organizing code into modular components

This project provided practical experience with Python's object-oriented features and demonstrated how inheritance and polymorphism can create flexible, maintainable code structures.